

Exploring and Improving Fault Localization in LLM Inference Engines

Tianhao Gu

02.2026–08.2026

Software Engineering & AI (SEAI)

Examiner: Prof. Dr. Chunyang Chen

Supervisor: Dr. Mark Huasong Meng

FROM FAILURE TO FAULT LOCATION

•  Issue report from GitHub

inference-engine /

└ server /

└ runner /

└ backend /

└ **CMakeLists.txt** ←

CONTENTS

Outline

- 01 Research Background**
Why is fault localization in LLM inference engines important and challenging?

- 02 RQ1 — Are the existing Fault Localization (FL) methods feasible?**
IRFL vs LLM-agent-based FL

- 03 RQ2 — Why do LLM-agent-based methods fail in some cases?**
An exploratory look at failed trajectories

- 04 RQ3 — How does ClueFL (our agent) improve the baselines?**
Can domain knowledge and the designed workflow improve fault localization for LLM inference engines?

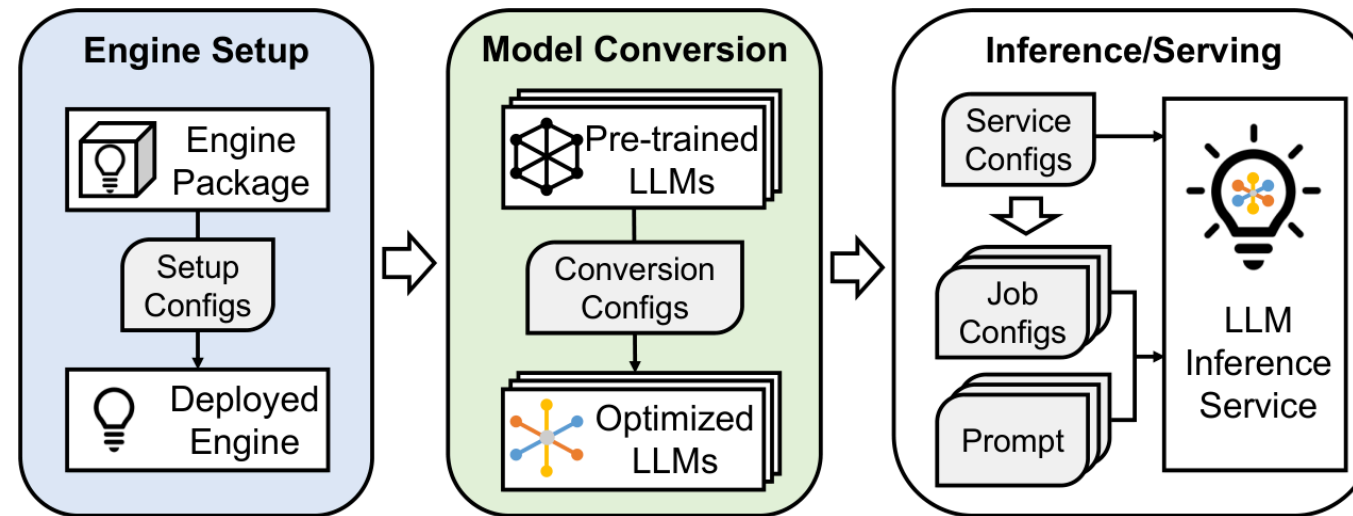
- 05 Contributions and Conclusions**
Three contributions and the answers to RQ1–RQ3

PART I

Research Background

Why is fault localization in LLM inference engines important and challenging?

How is an LLM deployed?



Workflow of LLM deployment^[1]

01

Engine setup

Install or compile the inference engine and verify hardware compatibility.

02

Model conversion

Convert, optimize, and package the model for the selected engine and environment.

03

Inference / serving

Load the model, process requests, and generate responses.

The LLM inference engine is responsible for all three phases, from engine setup to serving.

[1] Liu et al. 2026 · “A First Look at Bugs in LLM Inference Engines” · Fig. 1 ↗

Inference engines are everywhere, and so are their bugs.

VOLUME · 2025

16,500+

GitHub issues in one year across Ollama, vLLM, llama.cpp, SGLang [3]

Bugs arrive by the thousands.

Ollama alone: 3,653 issues in 2025, while serving 8.9M developers.

IMPACT

1,000+

public Ollama servers open to takeover through one bug [2]

One bug can reach production models.

One bug in Ollama let anyone on the internet take control of the server and steal or change its models. vLLM alone has shipped 70 security fixes since 2025, 9 rated critical.

FIX EFFORT

30%

of 929 real bugs take over a month to fix [1]

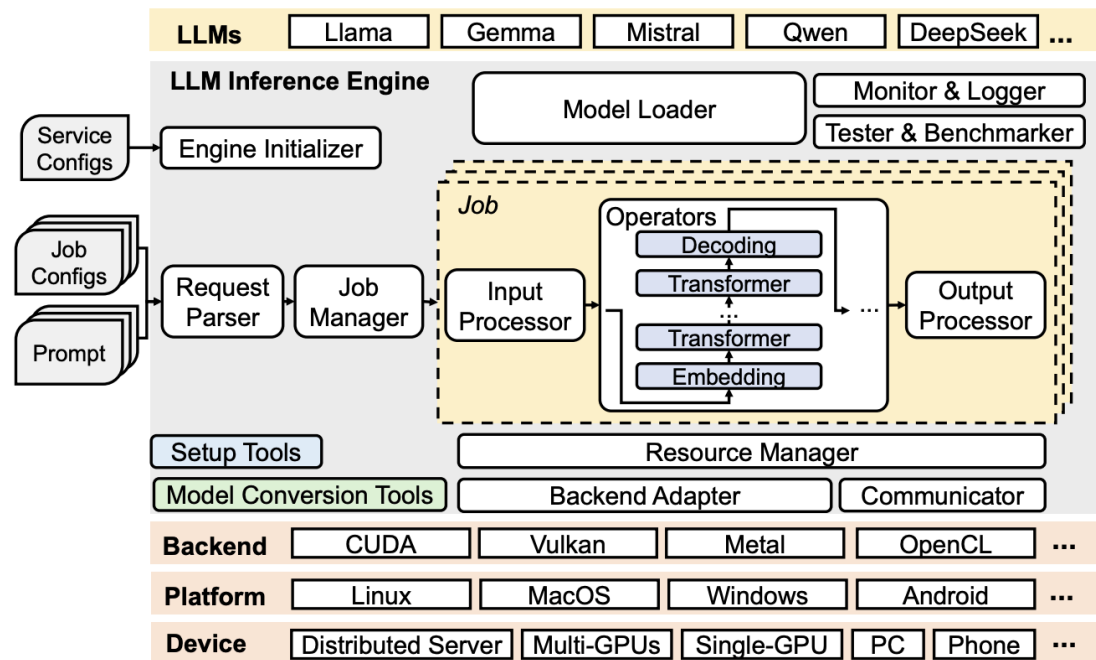
And fixing them is slow.

Half take longer than a week; system hangs average 65 days to resolve.

Many bugs, real damage, slow fixes. Why is this software so hard to get right?

01 / BACKGROUND

LLM inference engines have complex infrastructure.

Architecture overview^[1]

The complex infrastructure coordinates request handling, model execution, resource management, backend adapters, and heterogeneous devices.

MODULE STRUCTURE

Interdependent modules across the serving stack

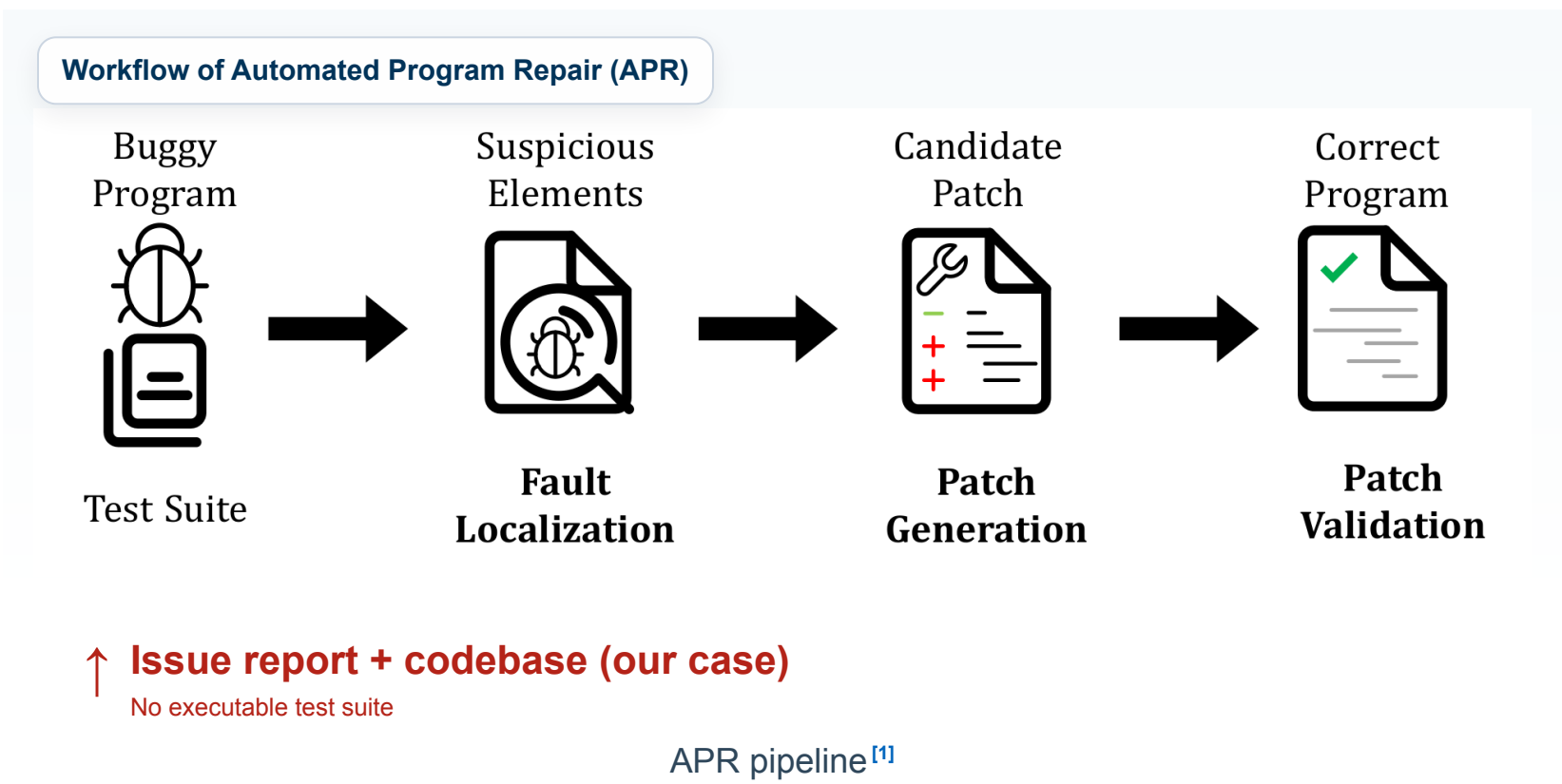
→ *in which module does the fault actually lie in?*

LIFECYCLE SCOPE

Spans setup, conversion, and LLM inference

→ *which phase is the starting point of the failure?*

What is Fault Localization?



FAULT LOCALIZATION

“Given the bug report and codebase, FL techniques identify buggy code elements (e.g., files or functions).”

Definition from Zhou et al. 2025^[1].

IMPORTANCE

Failure of FL can directly cause failure of the whole APR process.

APR needs to localize the bug before patch generation and validation.

[1] Definition: Zhou et al. 2025 · “Taming System Complexity: Demystifying SE Agents in Diagnosing Linux Kernel Faults” · APR pipeline: Zhang et al. 2024 · “A Systematic Literature Review on LLMs for Automated Program Repair” · Fig. 1

Why is FL hard for LLM inference engines?

LARGE-SCALE REPOSITORY

1,254

tracked files · 324k LOC

The candidate space is too large for a single LLM context.

The repository cannot be fed directly into one prompt, so file-level localization must first narrow the search space before detailed reasoning begins.

HETEROGENEOUS BUG TYPES

28

root-cause categories ^[1]

Bugs often have many possible root causes.

The same symptom can stem from configuration, runtime, I/O, or functional logic, so the FL approach must account for the many different ways a bug can arise.

REPRODUCTION CONSTRAINTS

0

executable test suite

Reliable reproduction evidence is often missing.

Many cases depend on specific hardware, drivers, models, or deployment environments, and issues often lack failing tests, complete traces, or stable reproduction steps.

Research questions.

RQ1

Feasibility Exploration — Despite the aforementioned challenges, are representative IR-based and LLM-agent-based approaches feasible for execution-free file-level localization of the issues in the LLM inference engines?

Assessment: baseline feasibility

RQ2

Failure Explainability — Why do LLM agents miss the repair file in LLM inference engines? Can the failures be characterized from the archived trajectories?

Analysis: observed failure modes

RQ3

Domain-Specific Improvement — Can incorporating domain-specific knowledge and a designed workflow improve fault localization for LLM inference engines?

Method proposal: ClueFL

CONTENTS

Outline

- 01 **Research Background**
Why is fault localization in LLM inference engines important and challenging?
- 02 **RQ1 — Are the existing Fault Localization (FL) methods feasible?**
IRFL vs LLM-agent-based FL
- 03 **RQ2 — Why do LLM-agent-based methods fail in some cases?**
An exploratory look at failed trajectories
- 04 **RQ3 — How does ClueFL (our agent) improve the baselines?**
Can domain knowledge and the designed workflow improve fault localization for LLM inference engines?
- 05 **Contributions and Conclusions**
Three contributions and the answers to RQ1–RQ3

NEXT

PART II

RQ1 · Feasibility

Are the existing FL methods feasible?

Why use Ollama?

01 · REAL-WORLD ADOPTION

Production-scale usage

8.9M^[1]

developers served

Issues are filed by real users against a live, actively maintained engine.

→ Failures are real, not synthetic, so FL results transfer to practice.

02 · FULL PIPELINE

Covers all three deployment phases

SETUP

build & deploy, hardware backends

CONVERSION

model acquisition, model management

INFERENCE

API serving, request orchestration, resource scheduling, inference runners

→ A bug can originate in any phase, so the benchmark spans the whole failure space.

03 · HETEROGENEOUS CODEBASE

Multi-language, multi-module repository

Go

C / C++

TypeScript

config & build files

Runtime logic, native inference kernels, tooling, and configuration live side by side in one repository.

→ The fault can hide in any language or artifact, so FL must reason across all of them.

Building OllamaFLBench.

1

Issue collection

Mine closed Ollama bug reports that were actually fixed by a merged pull request.

2

Fault location identification

Reconstruct the pre-fix snapshot and derive the exact repair location from the accepted fix.

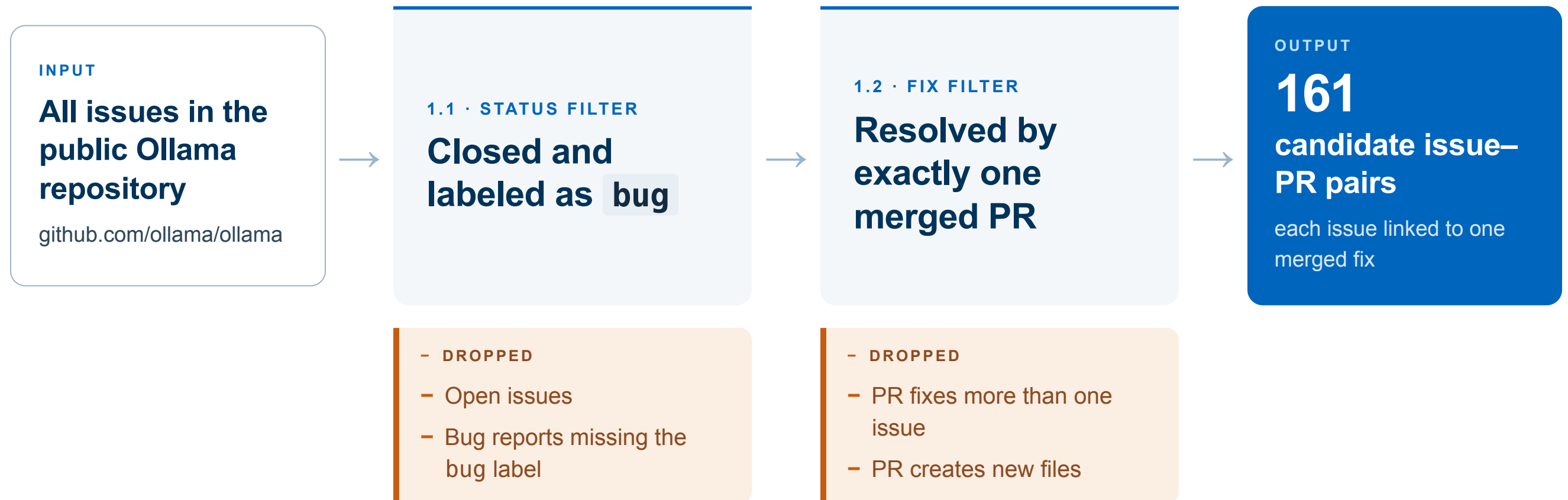
3

Manual inspection

Manually check every candidate, and recover missed ones with LLM assistance, so only clean, single-location instances remain.

Step 1 — Issue collection.

Mine closed, bug-labeled Ollama issues that were actually fixed by a merged pull request.



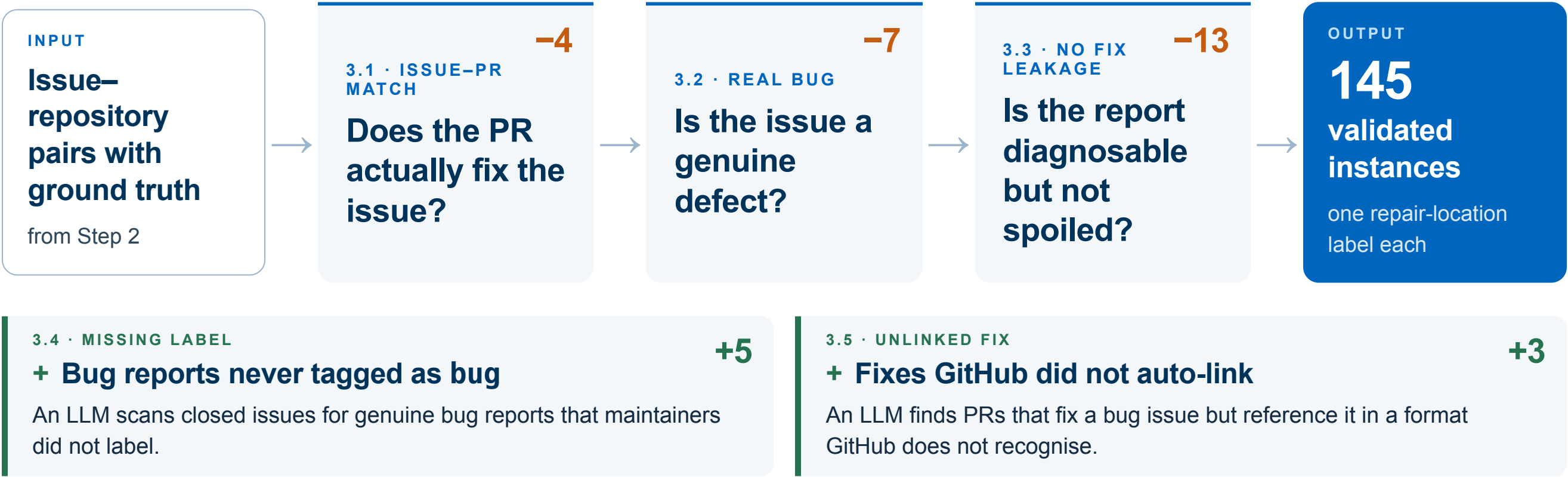
Step 2 — Fault location identification.

Take the repository state before the fix as input; the single file modified by the developer patch is the ground truth.



Step 3 — Manual inspection.

Manually check every candidate so only clean, single-label instances remain.



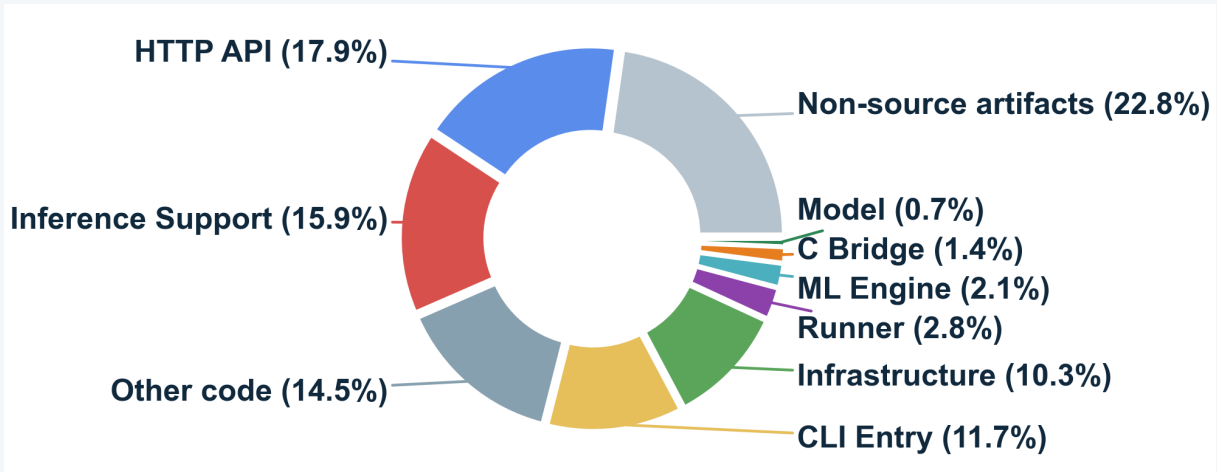
Characteristics of OllamaFLBench.

GROUND-TRUTH FILE TYPE **Source code vs. non-source, 145 repair files**

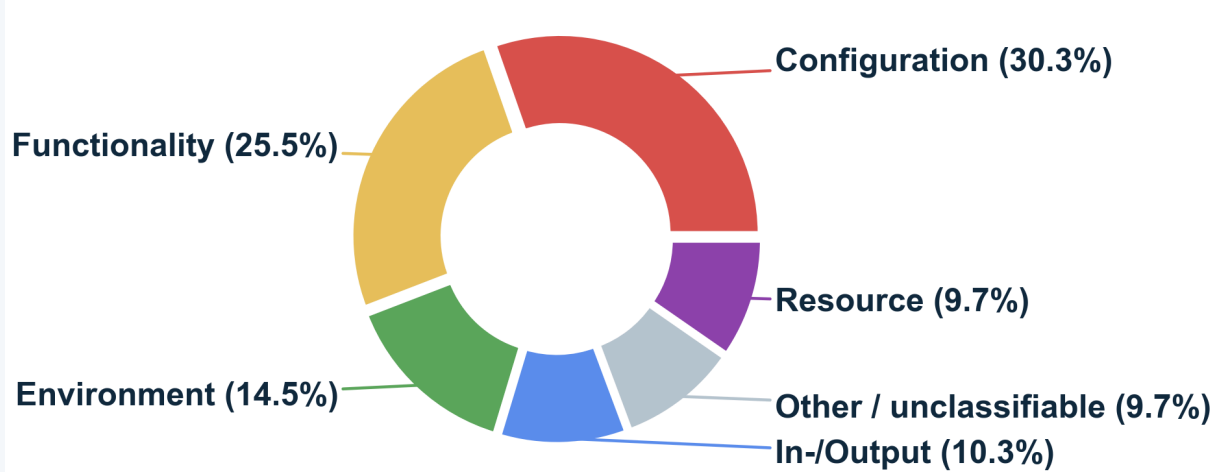


Source code · 112 Go 91 · shell 11 · C/C++ 6 · TypeScript 2 · PowerShell 2 **Non-source · 33** docs 19 · build files 8 · config/CI 5 · patch 1

WHERE IS THE FIX? **Repair files by software layer^[1]**



WHY DOES IT FAIL? **Root causes by family^[1]**



[1] Liu et al. 2026 · “A First Look at Bugs in LLM Inference Engines” ↗ · Dataset: github.com/tianhaogutum/ollama_evaluation_dataset ↗ · Thesis, Section 4.1, Table 4.1, Appendix A.2

Overview of existing fault-localization techniques.

✗ NOT APPLICABLE TO OLLAMAFLBENCH

✓ BASELINES EVALUATED IN RQ1

LLM-based FL

e.g., LLMAO, AutoFL

Feeds the bug report and large spans of source code directly into one LLM call to score suspicious files or lines.

[5]

Excluded — a full repository
✗ exceeds the single-pass context budget

SBFL / MBFL

Spectrum- and mutation-based FL

SBFL

Correlates pass/fail test-coverage spectra with suspicious statements.

MBFL

Compares mutant-kill patterns between passing and failing tests.

[6] [7]

✗ Excluded — requires an executable test suite

IRFL

Information-Retrieval Fault Localization

Ranks repository files by text and structural similarity between the bug report and the source code — no execution needed.

[1] [2]

✓ Feasible — repository text only

LLM-agent-based FL

e.g., SWE-agent, AutoCodeRover

An LLM autonomously searches, opens, and reasons over the repository, then proposes candidate files from the issue alone.

[3] [4]

✓ Feasible — no execution required

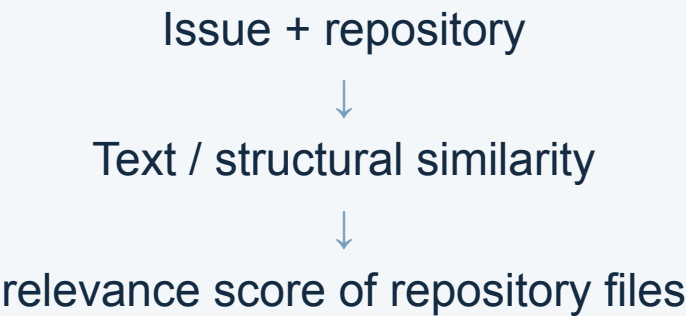
[1] Zhou et al. 2012 · “Where Should the Bugs Be Fixed? More Accurate IR-Based Bug Localization...” · [2] Saha et al. 2013 · “Improving Bug Localization Using Structured Information Retrieval” · [3] Yang et al. 2024 · “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering” · [4] Zhang et al. 2024 · “AutoCodeRover: Autonomous Program Improvement” · [5] Wu et al. 2023 · “Large Language Models in Fault Localisation” · [6] Jones & Harrold 2005 · “Empirical Evaluation of the Tarantula Automatic Fault-Localization Technique” · [7] Papadakis & Le Traon 2015 · “Metallaxis-FL: Mutation-Based Fault Localization”

Studied Baselines.

IRFL REPRESENTATIVE METHODS

BugLocator^[1] / BLUiR^[2]

Multilingual adaptations used in this thesis

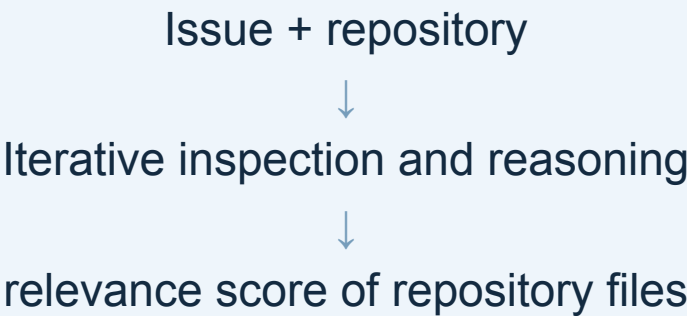


Strength	Deterministic retrieval
Challenge	Failure–fault location vocabulary mismatch

LLM-AGENT-BASED FL REPRESENTATIVE METHOD

SWE-agent^[3]

GPT-5.2 / DeepSeek V4 Pro configurations



Strength	Semantic, multi-step exploration
Challenge	Computation and exploration failures

PROTOCOL IRFL methods are deterministic given the same input. SWE-agent is run **twice per case** and the reported results are averaged over the two runs.

[1] Zhou et al. 2012 · “Where Should the Bugs Be Fixed? More Accurate IR-Based Bug Localization...” · [2] Saha et al. 2013 · “Improving Bug Localization Using Structured Information Retrieval” · [3] Yang et al. 2024 · “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering”

Fault-location ranking results.

METHOD	TOP-1		TOP-3		TOP-5		MRR@5	
IRFL · INFORMATION-RETRIEVAL BASELINES								
BugLocator	23.45%	↓ 41.38	42.07%	↓ 37.93	48.97%	↓ 40.00	0.3320	↓ 0.4068
BLUiR	21.38%	↓ 43.45	38.62%	↓ 41.38	52.41%	↓ 36.56	0.3187	↓ 0.4200
LLM-AGENT-BASED · SWE-AGENT								
SWE-agent GPT-5.2	51.03%	↓ 13.80	72.41%	↓ 7.59	75.17%	↓ 13.80	0.6138	↓ 0.1249
SWE-agent DeepSeek V4 Pro	64.83%	94 / 145	80.00%	116 / 145	88.97%	129 / 145	0.7387	best

Top-k: fault location appears in the first k predictions. MRR@5: mean reciprocal rank, scored zero beyond rank 5. ↓: gap from the column best.

FINDING 1

Feasible despite the challenges

Even with the vocabulary-mismatch and exploration challenges noted above, both IRFL and LLM-agent-based methods are feasible for execution-free fault localization on OllamaFLBench.

FINDING 2

Agents outperform IRFL

Across the evaluated backbones, LLM-agent-based methods substantially outperform IRFL methods overall.

FINDING 3

Room to improve

Considerable room for improvement remains; understanding how requires analyzing per-case exploration trajectories.

CONTENTS

Outline

- 01 **Research Background**
Why is fault localization in LLM inference engines important and challenging?
- 02 **RQ1 — Are the existing Fault Localization (FL) methods feasible?**
IRFL vs LLM-agent-based FL
- 03 **RQ2 — Why do LLM-agent-based methods fail in some cases?**
An exploratory look at failed trajectories
- 04 **RQ3 — How does ClueFL (our agent) improve the baselines?**
Can domain knowledge and the designed workflow improve fault localization for LLM inference engines?
- 05 **Contributions and Conclusions**
Three contributions and the answers to RQ1–RQ3

NEXT

PART III

RQ2 · Failure Analysis

An exploratory look at failed trajectories

Three failure modes observed in the archived trajectories.

Why do LLM agents miss the repair file? Reading the archived trajectories reveals three observed failure modes.

01

Skipped **non-code files**

The fix is in a doc, config, or build script, but the agent only searches source files and never opens it.

02

Settled on **a similar file**

The agent reaches the right package, builds a plausible story around a similar file, and never opens the one that actually needs the fix.

03

Stopped at **a software-layer boundary**

The agent traces the request within one layer, but stops where the Go runner hands off to the C/C++ engine; the root cause sits in the next layer.

Skipped non-code files.

The fix is in a doc, config, or build script, but the agent only searches source files and never opens it.

A search limited to source extensions can never surface build or CI files. A slowdown after an upgrade can come from how a release is built, not only from runtime code.

CASE 109

Repair file:

`.github/workflows/release.yaml`

The fix is in the release workflow, but GPT-5.2 keeps reading thread and scheduling code and ranks the workflow only at rank 10.

1

Reads the issue

CPU inference on a 2-socket AMD EPYC 9654 slows after updating Ollama, suggesting a thread-count or scheduling regression.

2

Searches only source files

All keyword searches (`GetOptimalThreadCount` , `defaultThreads` , `--threads` , `SCHED_SPREAD`) are limited to `.go` and other source files.

3

Reads runtime code

Opens `llm/server.go` and `server/sched.go` , and builds a thread-count explanation around `--threads` and scheduling.

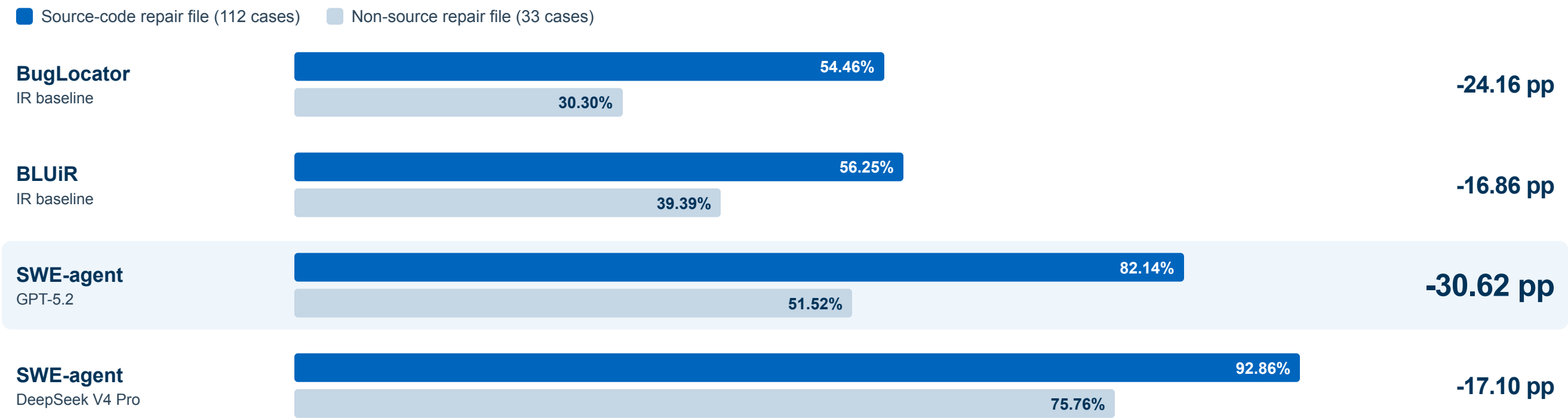
✗

Never opens the release pipeline

`.github/workflows/release.yaml` is never read; it only appears at rank 10 from a few-shot hint, so the CI-side fix cannot win.

Skipped non-code files.

Top-5 accuracy by artifact type of the ground-truth repair file, from Table 4.6.



Settled on a similar file.

The agent reaches the right package, settles on a similar file, and never opens the one that actually needs the fix.

Strong local evidence narrows the search so much that a neighbouring file never enters the context. This is an evidence-allocation problem, not a repository-size problem.

CASE 126

Repair file: `discover/cpu_windows.go`

Both GPT-5.2 and DeepSeek V4 Pro rank it outside their Top-5.

- 1 Reads the issue**
Windows startup fails on a machine with no GPU installed. The crash log is full of GPU-discovery messages.
- 2 Follows the GPU evidence**
Opens `discover/gpu_cuda.go` and `discover/gpu_rocm.go` to check the GPU-detection logic.
- 3 Expands within the same branch**
Moves on to `discover/runner.go`, the scheduler, the runner lifecycle and CUDA backend initialisation.
- 4 Keeps re-reading GPU files**
Both agents return to the files named in the log instead of asking which neighbouring path is still unexplored.
- ✗ Never opens the CPU startup path**
`discover/cpu_windows.go` sits in the same directory, yet stays outside the explored context in both runs.

Stopped at a software-layer boundary.

The agent stops where the Go runner hands off to the C/C++ engine; the root cause sits in the next layer.

When the fix crosses a language or process boundary, an agent that stops at that boundary misses it. The expected behaviour is to follow the state to where it first becomes invalid, not to dig deeper in one software-layer.

CASE 86

Repair file: `llm/ext_server/server.cpp`

GPT-5.2 misses it. DeepSeek V4 Pro crosses the boundary and ranks it first.

SWE-AGENT · GPT-5.2 MISS

- 1

Reads the issue
A streaming response is missing its final token.
- 2

Traces the Go side
`server/routes.go` , `llm/server.go` ,
`api/client.go` .
- 3

Blames Go-side scanning
Attributes the loss to the scanning loop and stream ordering, although the Go logic looks complete.
- ×

Stops at the language boundary
Never enters the native response producer where the token is actually dropped.

SWE-AGENT · DEEPSEEK V4 PRO HIT

- 1

Reads the issue
A streaming response is missing its final token.
- 2

Traces the Go side
Follows the same Go files as GPT-5.2.
- 3

Crosses into native code
Follows the response one hop further, into
`llm/ext_server/server.cpp` .
- ✓

Finds where the token is dropped
Checks the token and final-response path and ranks the file first.

CONTENTS

Outline

- 01 **Research Background**
Why is fault localization in LLM inference engines important and challenging?
- 02 **~~RQ1 — Are the existing Fault Localization (FL) methods feasible?~~**
~~IRFL vs LLM-agent-based FL~~
- 03 **~~RQ2 — Why do LLM-agent-based methods fail in some cases?~~**
~~An exploratory look at failed trajectories~~
- 04 **RQ3 — How does ClueFL (our agent) improve the baselines?**
Can domain knowledge and the designed workflow improve fault localization for LLM inference engines?
- 05 **Contributions and Conclusions**
Three contributions and the answers to RQ1–RQ3

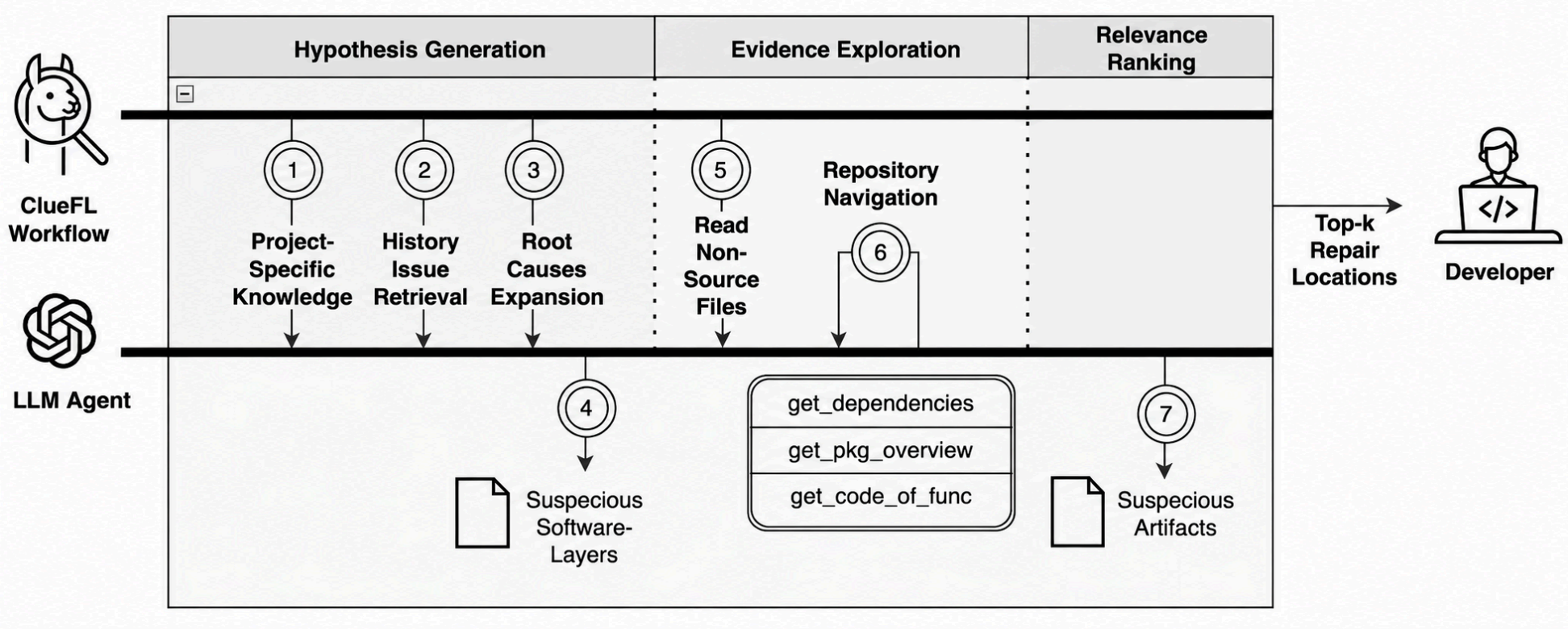
NEXT

PART IV

RQ3 · Improvement

How does ClueFL (our agent) improve baselines?

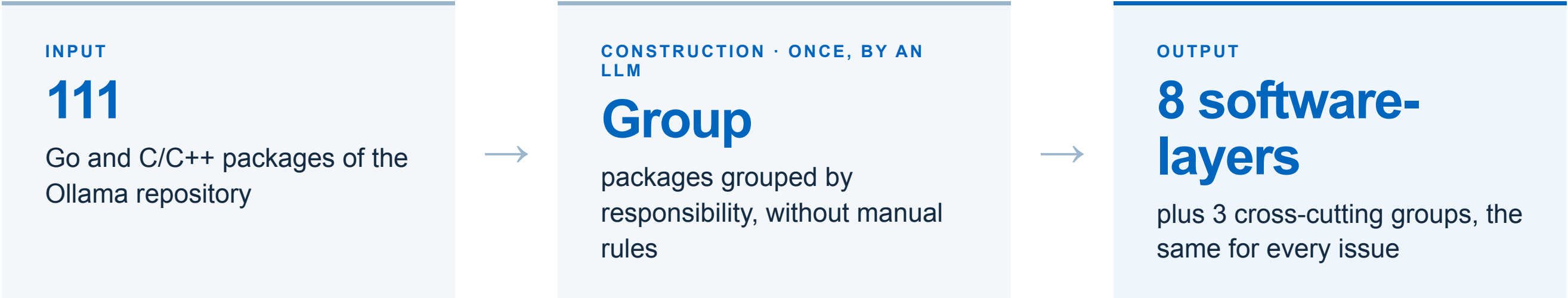
ClueFL: layer-aware, evidence-guided file ranking.



ClueFL framework

Step 1 — Software-layer knowledge.

Before searching, the agent receives a map of the repository's software-layers.



ABLATION The software-layer map contributes **+4.14 pp** Top-1 with GPT-5.2 and **+2.07 pp** with DeepSeek V4 Pro: the largest contribution of all four components.

Step 2 — Historical issue retrieval.

Past issues that resemble the current one, and the files that fixed them, are given to the agent as hints.

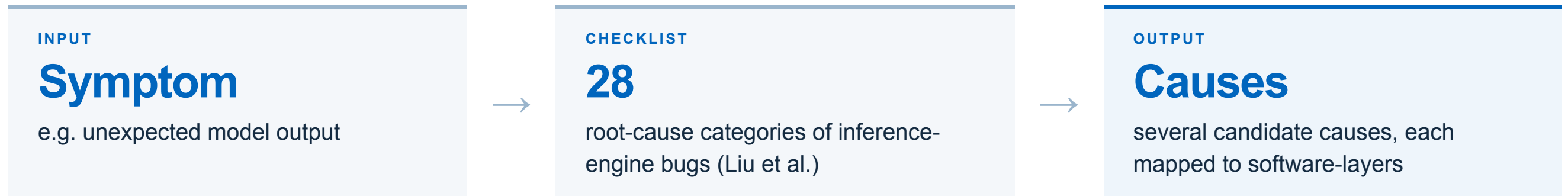


Example (PR 13445). A near-identical past issue pointed to `gemma3/model_text.go`, which the agent ranked first. The baseline searched tokenization code and missed it.

ABLATION Historical retrieval contributes **+2.76 pp** Top-1 with GPT-5.2 and **+1.38 pp** with DeepSeek V4 Pro.

Step 3 — Root-cause expansion.

Before searching, the agent lists several possible causes instead of committing to the first one.



Why. RQ2 showed that agents often fix on one explanation early and stop the causal chain there.

Example (PR 6905). The Windows GUI build runs slower than the CLI. The baseline assumed thread scheduling and ranked the true file 6th. With the checklist, the agent also considered an environment mismatch and ranked `llm/llm_windows.go` first.

ABLATION

The checklist contributes **+2.07 pp** Top-1 with GPT-5.2 and **+0.69 pp** with DeepSeek V4 Pro: the smallest contribution, but positive on both backbones.

Repository navigation tools.

deps <package_name>

Which packages a package depends on, and which depend on it.

pkg_summary <package_name>

Files, types, and function signatures of a package, without the source.

func <file> <function>

The source code of a function.

SEARCH ORDER

Software-layer



Package



File



Function

grep and find remain available

ABLATION

The tools contribute **+3.45 pp** Top-1 with GPT-5.2 and **+1.38 pp** with DeepSeek V4 Pro: the second-largest contribution.

Effect of domain guidance on fault-location ranking.

METHOD			TOP-1		TOP-3		TOP-5		MRR@5
GPT-5.2 BACKBONE									
SWE-agent	baseline	51.03%	74 / 145	72.41%	105 / 145	75.17%	109 / 145	0.6138	
ClueFL	ours	58.62%	↑ 7.59	75.86%	↑ 3.45	81.38%	↑ 6.21	0.6767	↑ 0.0629
DEEPSEEK V4 PRO BACKBONE									
SWE-agent	baseline	64.83%	94 / 145	80.00%	116 / 145	88.97%	129 / 145	0.7387	
ClueFL	ours	67.59%	↑ 2.76	82.76%	↑ 2.76	88.97%	→ 0.00	0.7583	↑ 0.0195

Matched pairs: same backbone, issues, pre-fix snapshots, and scorer; only the localization configuration changes. ↑: gain over the matched baseline in percentage points. One run per instance.

FINDING 1

Better early ranking on both backbones

Top-1 rises by **+7.59 pp** with GPT-5.2 (74 → 85 cases) and **+2.76 pp** with DeepSeek V4 Pro (94 → 98 cases); MRR@5 improves for both.

FINDING 2

More gain where the baseline is weaker

GPT-5.2 improves at every cutoff, including **+6.21 pp** Top-5. DeepSeek starts at 88.97% Top-5, so its gain is concentrated in ordering rather than coverage.

What each component contributes.

CONFIGURATION	TOP-1		TOP-3		TOP-5		MRR@5	
GPT-5.2 BACKBONE								
Full ClueFL	58.62%		75.86%		81.38%		0.6767	
– Software-layer knowledge	54.48%	+4.14	73.79%	+2.07	77.93%	+3.45	0.6445	+0.0322
– Navigation toolbox	55.17%	+3.45	73.79%	+2.07	78.62%	+2.76	0.6490	+0.0277
– Historical retrieval	55.86%	+2.76	74.48%	+1.38	79.31%	+2.07	0.6531	+0.0236
– Root-cause taxonomy	56.55%	+2.07	75.17%	+0.69	80.00%	+1.38	0.6605	+0.0162
DEEPSEEK V4 PRO BACKBONE								
Full ClueFL	67.59%		82.76%		88.97%		0.7583	
– Software-layer knowledge	65.52%	+2.07	80.69%	+2.07	88.28%	+0.69	0.7420	+0.0163
– Navigation toolbox	66.21%	+1.38	81.38%	+1.38	88.28%	+0.69	0.7460	+0.0123
– Historical retrieval	66.21%	+1.38	81.38%	+1.38	88.97%	+0.00	0.7470	+0.0113
– Root-cause taxonomy	66.90%	+0.69	82.07%	+0.69	88.97%	+0.00	0.7520	+0.0063

Leave-one-component-out: each row removes one component, everything else unchanged. +x: contribution of that component, i.e. full ClueFL minus the ablated variant, in percentage points. Rows ordered by contribution to MRR@5.

CONTENTS

Outline

- 01 **Research Background**
Why is fault localization in LLM inference engines important and challenging?
- 02 **RQ1 — Are the existing Fault Localization (FL) methods feasible?**
IRFL vs LLM-agent-based FL
- 03 **RQ2 — Why do LLM-agent-based methods fail in some cases?**
An exploratory look at failed trajectories
- 04 **RQ3 — How does ClueFL (our agent) improve the baselines?**
Can domain knowledge and the designed workflow improve fault localization for LLM inference engines?
- 05 **Contributions and Conclusions**
Three contributions and the answers to RQ1–RQ3

NEXT

PART V

Contributions and Conclusions

The three contributions of this thesis.

Three contributions.

SYSTEMATIC EVALUATION

01

First execution-free evaluation of file localization for LLM inference-engine bugs

Representative IR-based (BugLocator, BLUIR) and LLM-agent-based (SWE-agent) approaches compared on the same instances in **effectiveness, cost, backbone models, and failure behavior**.

88.97% of repair files within Top-5 for the best agent configuration, without running a single test

REUSABLE DATASET

02

OllamaFLBench: a repository-derived benchmark, publicly released

145 validated issue-fix pairs with pre-fix snapshots and unambiguous, patch-derived single-file repair labels spanning eligible **source and non-source** artifacts.

145 instances · 1 verified repair file each · 112 source, 33 non-source

DOMAIN-SPECIFIC FRAMEWORK

03

ClueFL: domain knowledge and a structured workflow improve file ranking

An 8-layer architecture map, historical fixes, a 28-category root-cause checklist, and three navigation tools organized into hypothesis generation, evidence exploration, and relevance ranking.

+7.59 Top-1 points over SWE-agent with GPT-5.2, +2.76 with DeepSeek V4 Pro; every component contributes

Research questions and answers.

RQ1

ARE EXISTING FAULT-LOCALIZATION METHODS FEASIBLE?

Yes. IR-based and LLM-agent-based methods both give useful file rankings without executing code. Both agents beat BugLocator and BLUiR at every cutoff; **DeepSeek V4 Pro** reaches 64.83% Top-1 and 88.97% Top-5, at a higher cost in latency and tokens.

RQ2

WHY DO LLM AGENTS MISS THE REPAIR FILE?

Not at random. Reading the failed trajectories reveals three observed failure modes: **skipped non-code files**, **settled on a similar file**, and **stopped at a software-layer boundary**.

RQ3

CAN DOMAIN KNOWLEDGE AND A DESIGNED WORKFLOW HELP?

Yes. ClueFL improves the ranking with both backbones: Top-1 **+7.59 pp** with GPT-5.2 and **+2.76 pp** with DeepSeek V4 Pro, with the larger gain for the weaker backbone.

THANK YOU

Thank you for **your** attention.

Questions and discussion are very welcome.

MASTER'S THESIS

Exploring and Improving Fault Localization in LLM Inference Engines

Tianhao Gu · Software Engineering & AI (SEAI)

Examiner: Prof. Dr. Chunyang Chen

Supervisor: Dr. Mark Huasong Meng

DATASET

github.com/tianhaogutum/ollama_evaluation_dataset